



# Blueprint Integrasi Algoritma

Simulasi UTS Praktikum: Sistem Pengelolaan Gaji Karyawan

IP0 Framework

Modular Design

Big-0 Evaluation

# Spesifikasi Operasional UTS

## Mission Briefing: Target & Batasan Sistem



### Input Data

Terima input dinamis berupa jumlah karyawan dan array berisi nominal gaji (dalam jutaan rupiah).



### Mesin Pengurut

Wajib menggunakan algoritma Bubble Sort. Tulis logika secara manual tanpa fungsi bawaan.



### Mesin Pencari

Gunakan algoritma Linear Search untuk menemukan posisi nominal gaji spesifik secara berurutan.



### Audit Sistem Akhir

Tampilkan hasil urutan, koordinat indeks, total jumlah perbandingan/langkah, dan evaluasi dominasi Big-O.

# Arsitektur Sistem Berbasis IPO



# Transisi Paradigma: Monolitik ke Modular

Prinsip Isolasi: Jangan pernah menggabungkan proses Sorting dan Searching dalam satu fungsi.

## Monolitik (Dihindari)



Satu blok kode raksasa. Sulit dilacak, rentan error.

## Modular (Standar UTS)

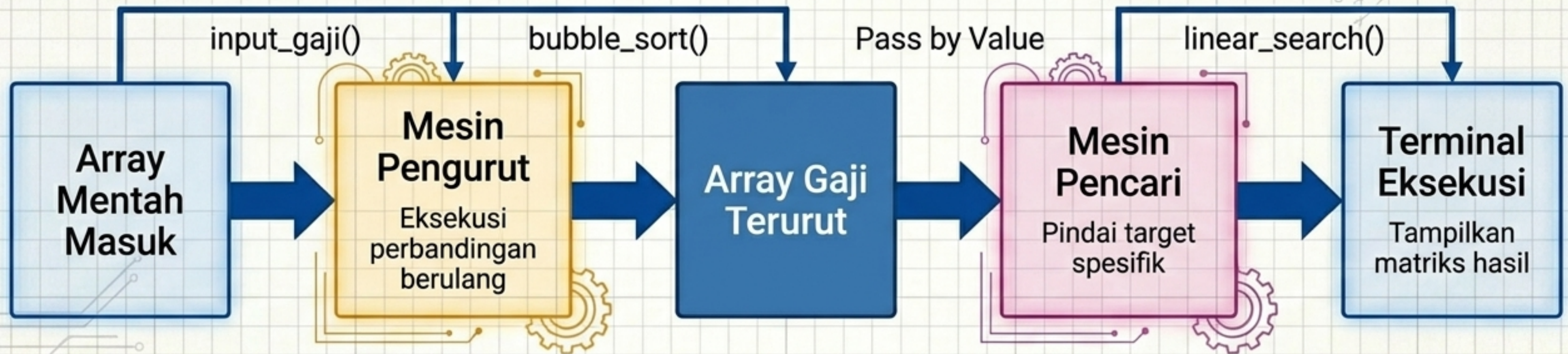


Setiap sub-masalah memiliki ruang eksekusi mandiri.

- Dekomposisi Sistem:**
- 1. input\_gaji()
  - 2. bubble\_sort(arr)
  - 3. linear\_search(arr, target)
  - 4. tampilkan\_data(arr)
  - 5. main()

# Pipeline Integrasi Algoritma

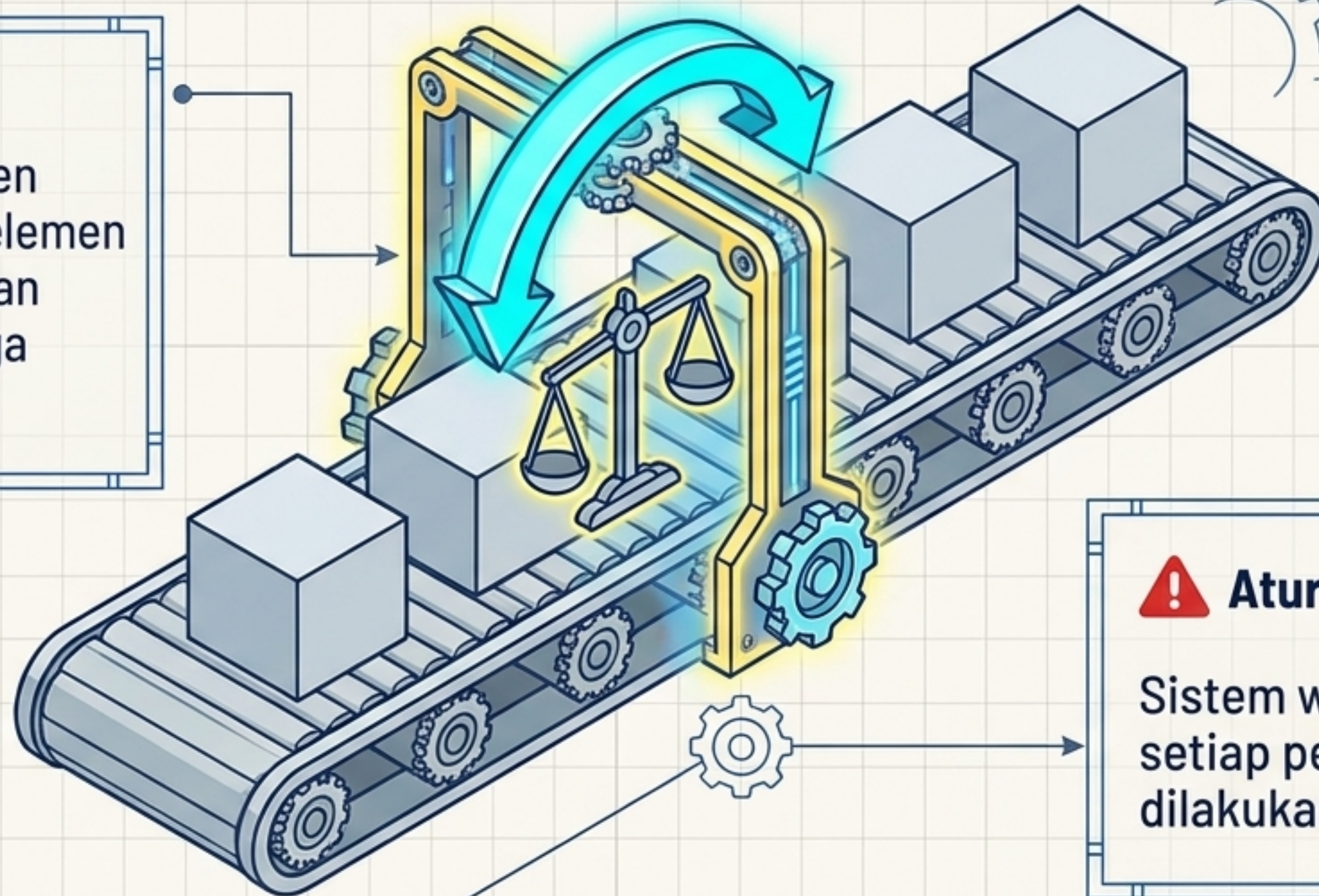
The Master Plan: Alur Transformasi Data



# Mesin Pengurut: Mekanika Bubble Sort

## Logika Dasar:

Bandingkan dua elemen bersebelahan. Geser elemen terbesar ke ujung kanan secara berulang hingga seluruh data terkunci.



## ⚠ Aturan UTS Kritis:

Sistem wajib menghitung setiap perbandingan yang dilakukan (saat  $j > j+1$ ).

# Anatomi Kode Modular: Bubble Sort

```
def bubble_sort(arr):  
    n = len(arr)  
    counter_urut = 0  
    for i in range(n):  
        for j in range(0, n-i-1):  
            counter_urut += 1  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr, counter_urut
```

## Loop Bersarang:

Penentu kompleksitas  $O(n^2)$ .

## Operasi Pertukaran:

Manipulasi array utama berdasarkan kondisi.

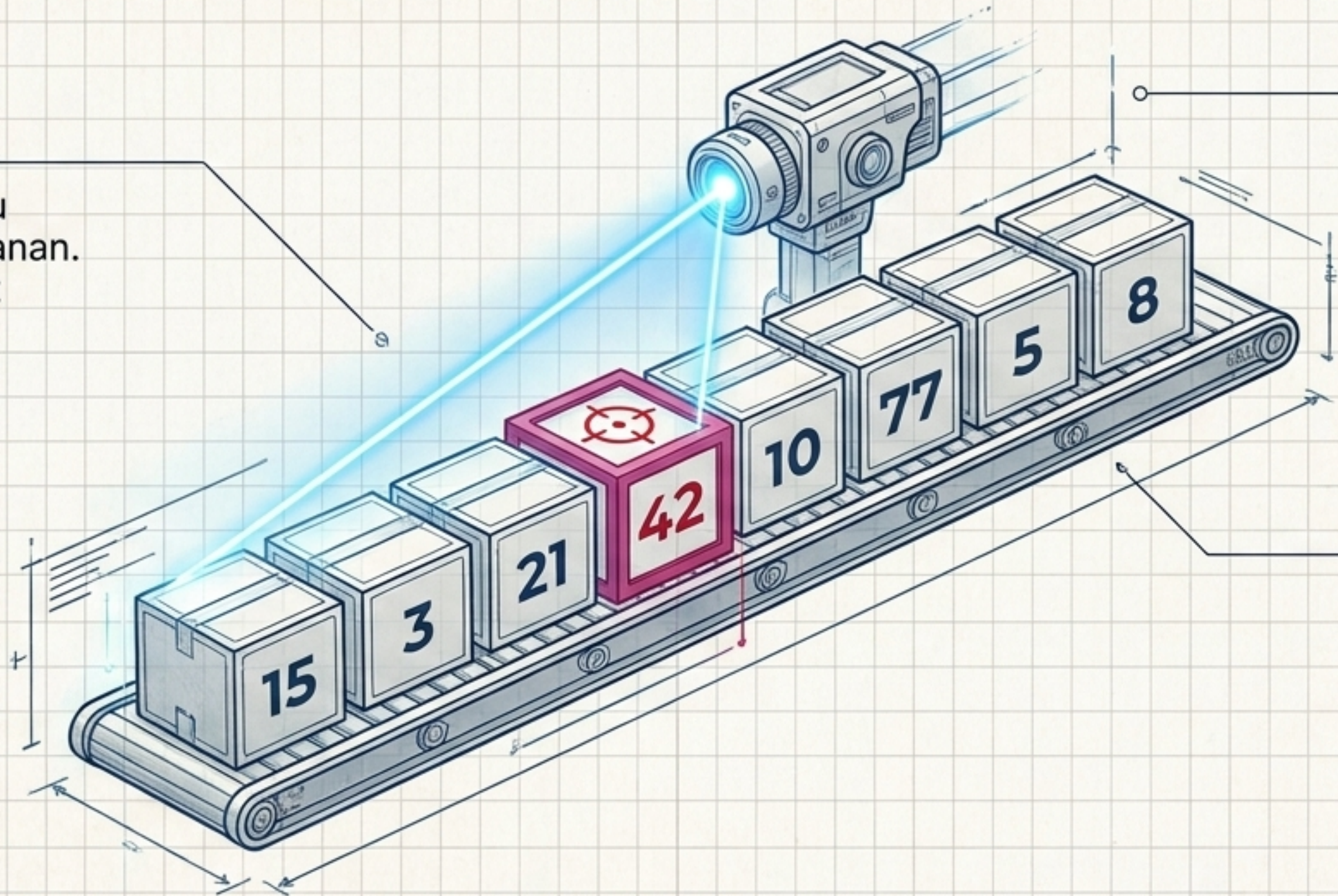
## [PENTING] Injeksi Counter:

Wajib diletakkan di dalam perulangan untuk melacak beban evaluasi performa.

# Mesin Pencari: Mekanika Linear Search

## Logika Dasar:

Memindai elemen satu per satu dari Kiri ke Kanan. Berhenti seketika saat target ditemukan.



## Karakteristik Operasional:

Sangat aman digunakan baik pada data acak maupun data yang sudah terurut.

Diwajibkan melacak jumlah langkah pemindaian.

# Anatomi Kode Modular: Linear Search

```
def linear_search(arr, target):  
    counter_cari = 0  
    for i in range(len(arr)):  
        counter_cari += 1  
        if arr[i] == target:  
            return i, counter_cari  
    return -1, counter_cari
```

## Loop Tunggal:

Menjamin eksekusi linear secara efisien  $O(n)$ .

## Injeksi Counter:

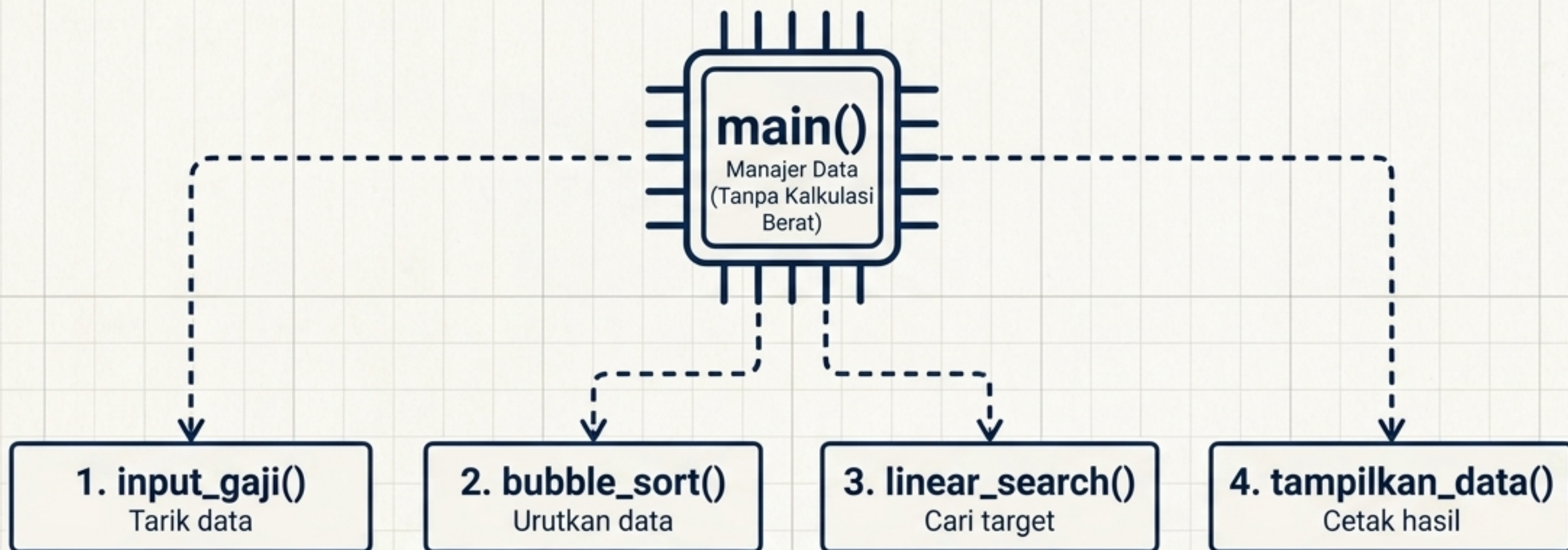
Diletakkan di setiap awal iterasi untuk menghitung total langkah pemindaian secara akurat.

## Kondisi Fallback:

Sistem wajib mengembalikan nilai -1 jika seluruh array habis dipindai dan target tidak eksis.

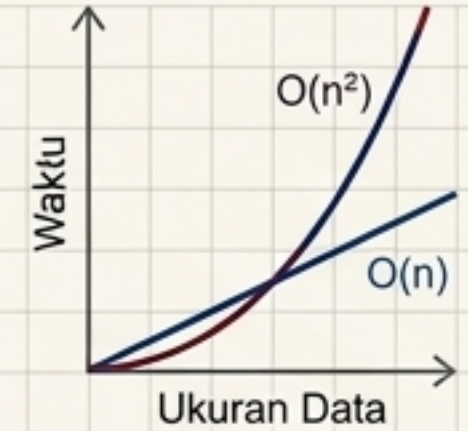
# Orkestrasi Sistem: Program Utama

Fungsi `main()` sebagai konduktor lalu lintas data



**Prinsip Utama:** Fungsi `main()` hanya memanggil fungsi-fungsi mandiri dan mengirimkan variabel (`arr`) secara berurutan antar modul.

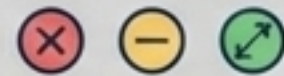
# Matriks Diagnostik Big-O (Evaluasi Sistem)



| Komponen Algoritma                     | Notasi Kompleksitas  | Karakteristik Eksekusi                                                                                                                                              |
|----------------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mesin Pengurut:</b><br>Bubble Sort  | $O(n^2)$ — Kuadratik | Menggunakan loop bersarang. Sangat rentan terhadap ledakan iterasi dan penurunan performa tajam pada himpunan data berukuran masif. Mendominasi beban kerja sistem. |
| <b>Mesin Pencari:</b><br>Linear Search | $O(n)$ — Linear      | Menggunakan loop tunggal. Kecepatan pencarian tumbuh secara stabil dan proporsional. Aman diimplementasikan untuk data skala kecil hingga menengah.                 |

# Hasil Eksekusi & Laporan Sistem

Blueprint Output Final Terminal



```
=== SISTEM PENGELOLAAN GAJI KARYAWAN ===  
"Data Gaji Terurut: [4, 7, 10, 15, 22] (dalam Juta Rp)  
"Pencarian: Gaji 15 Juta ditemukan pada indeks 3.  
  
=== ANALISIS SINGKAT ===  
"Algoritma sorting      : Bubble Sort  
"Kompleksitas sorting:  $O(n^2)$   
"Jumlah perbandingan   : 10  
"Algoritma searching   : Linear Search  
"Kompleksitas searching:  $O(n)$   
"Jumlah langkah pencarian: 4  
"Algoritma yang lebih dominan kompleksitasnya: Sorting ( $O(n^2)$ )"
```

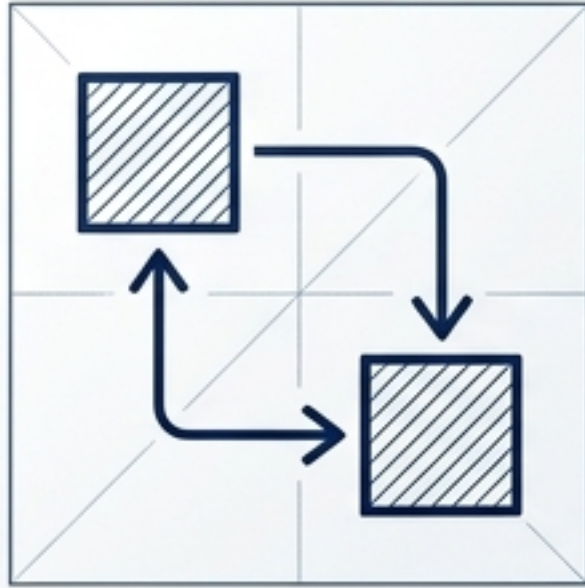
# Checklist Keselamatan UTS

4 Golden Rules untuk Mencegah Kegagalan Kritis



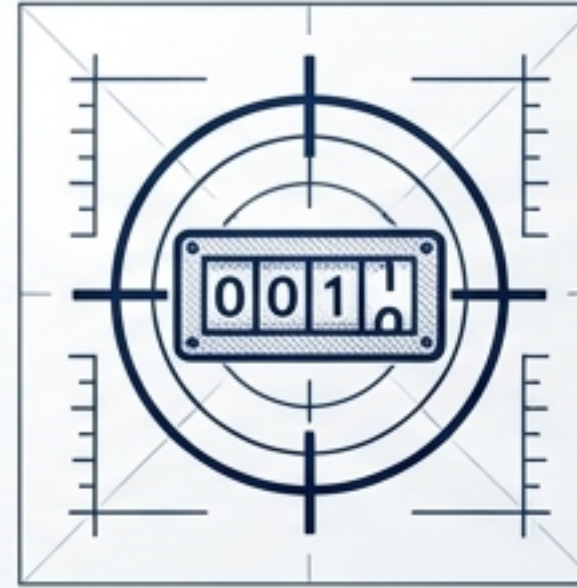
## 1. Larangan Fungsi Bawaan

Dilarang keras menggunakan fungsi instan seperti `.sort()` atau `.index()`. Semua algoritma wajib ditulis secara manual dari nol.



## 2. Isolasi Tanggung Jawab

Jangan menggabungkan proses pencarian ke dalam fungsi pengurutan. Modul harus berdiri sendiri.



## 3. Akurasi Audit Counter

Pastikan variabel counter di-injeksi tepat di dalam blok logika (loop/if) terdalam untuk akurasi pelacakan beban kerja.



## 4. Cetak Analisis Akhir

Jangan lupakan blok evaluasi Big-O di akhir terminal. Sistem logis yang benar tanpa laporan analisis tetap akan kehilangan poin krusial.